# EXPEDIENTES ALUMNOS

Vamos a diseñar una aplicación para guardar los expedientes académicos de alumnos. Tendremos las siguientes clases:

**Estudiante**

- Atributos: dni, nombre, apellidos, fnacimiento, direccion, localidad, email, telefono.
- Constructor, getters, setters, toString, equals(dni)

**NotasCurso**

- Atributos: etapaEducativa (ESO, Bachillerato, FP Básica, FP Medio, FP Superior), nombreCiclo (puede ser vacío), curso (1,2,3,4), **notas** (**HashMap**<String, Double>, guarda asignatura y nota), anio.
- Constructor, getters, setters, toString, equals (etapa, nombre, curso y año).
- Métodos sobre notas:
  - *addNota(String asignatura, Double nota)*
  - *deleteNota(String asignatura)*
  - *buscar(String asignatura)* que devuelve la nota de una materia.
  - *pintar(),* pinta las notas ordenadas por asignatura

**Expediente**

- Atributos: Estudiante estudiante, **HashSet** notas, activo (si o no)
- Constructor, getters, setters (los que hagan falta), toString, equals (estudiante).
- Métodos extra:
  - *addNotas(NotasCurso nc),* para añadir todas las notas de un curso al expediente de un alumno.
  - *mostrarNotas(int curso, String etapa),* que mostrará todas las notas de este estudiante para una etapa y un curso concreto.

**Centro educativo**

- Atributos: **TreeSet** expedientes, nombreCentro, direccion, localidad, telefono, email
- Constructor, getters, setters, toString, equals(nombreCentro).
- Métodos extra:
  - *addExpediente(Expediente e)*, para añadir un expediente al centro.
  - *buscarExpediente(String dni)*, que devolverá todas las notas de todos los cursos de ese alumno.

Prueba las clases (es laborioso, lo sé) añadiendo unos alumnos, con notas para un par de cursos (1º grado superior daw y 2º grado superior daw). Luego busca el expediente de uno de los alumnos y muéstralo correctamente.

# GESTIÓN BIBLIOTECA

Vamos a diseñar una aplicación para gestionar los libros prestados en una biblioteca, usando tanto Set como Map.

**Libro**

- Atributos: ISBN, título, autor, género, año de publicación.
- Métodos: Constructor, getters, setters, toString(), equals(ISBN), hashCode().

**Usuario**

- Atributos: DNI, nombre, apellidos, dirección, teléfono, email.
- Métodos: Constructor, getters, setters, toString(), equals(DNI), hashCode().

**Préstamo**

- Atributos: Usuario usuario, Libro libro, fecha de préstamo, fecha de devolución (puede ser null si el libro aún no ha sido devuelto).
- Métodos:
  - Constructor, getters, setters, toString(), equals(usuario, libro).
  - *devolverLibro*(), que actualiza la fecha de devolución con la fecha actual.
  - *estaActivo*(), que devuelve true si el libro aún no ha sido devuelto (fechaDevolucion == null).

**Biblioteca**

- Atributos:
  - HashMap<String, Libro> catalogo → Almacena libros por ISBN.
  - HashMap<String, Usuario> usuarios → Almacena usuarios por DNI.
  - TreeMap<Usuario, HashSet<Prestamo>> prestamos → Asocia cada usuario con un conjunto de préstamos.
  - Nombre de la biblioteca, dirección, teléfono, email.
- Métodos:
  - Constructor, getters, setters, toString(), equals(nombreBiblioteca).
  - *addLibro*(Libro l), para añadir libros al catálogo.
  - *addUsuario*(Usuario u), para registrar un nuevo usuario.

- *esLibroDisponible*(String isbn): se debe recorrer todos los préstamos de todos los usuarios (values), y ver si de algún préstamo activo coincide el libro.
- *prestarLibro*(String dni, String isbn), que crea un nuevo préstamo ***si el libro está disponible*.**
- *devolverLibro*(String dni, String isbn), que marca el libro como devuelto si existe un préstamo activo.
- *buscarPrestamosUsuario*(String dni), que devuelve todos los libros que ha pedido prestados un usuario.
- *listarLibrosDisponibles*(), que muestra todos los libros no prestados.

Para probar este ejemplo, crea 20 libros, 4 usuarios, y que cada usuario pida prestados al menos dos libros. Por último, llama al método buscarPrestamosUsuario de un usuario, y a listarLibrosDisponibles, para ver los libros que no están prestados.